Java DAY 5

OOPS

Topic : class, object, methods, constructors, Encapsulation

1. why oop exists

imagine you are building a banking app.

you have:

* account holder name
* account number
* balance
* deposit logic
* withdraw logic

you could write random variables and functions everywhere

But that becomes messy.

So java says:

put related data + behaviour together

This bundle is called an object.

mental model

think of a car

A car has:

- data: color, brand, speed
- actions: start, stop, acceleration

in java:

- class: blueprint of the car
- object: actual car built from blueprint.

2. class vs object

class

A class is a design/template/blueprint

it defines:

- what data something has
- what actions it can do

object

An object is a real instance created from class

Real world example

```
class student {
    String name;
    int age;
}
```

Real objects:

Student 1 = Thareesh, 21

Student 2 = Rahul, 19

same blueprint, Different actual values

```
Student S1 = new Student();
```

means:

Student → blueprint type

S1 → Reference name

new Student() → build a fresh, student object in memory.

why object matters

without objects code becomes scattered:

```
String SIN = "thareesh";
int SIA = 20;
```

Then for 100 students?

mess.

with oop:

```
Student s1 = new Student();
Student s2 = new Student();
Student s3 = new Student();
```

Now the structure is clean and scalable.

methods

method is just:

A named block of logic that does one job.

mental model

A method is like a button on a machine

press button:

* deposit money
* print bill
* start engine

in Code:

object has data

method acts on that data,

Ex:

```
Class calculator {

    int a;
    int b;
    void add() {
        System.out.println(a+b);
    }
}
```

Constructors

why Constructor exists

when you create an object, java needs a way to initialize it properly.

A Constructure is:

A special method that runs automatically when an object is created.

Ex:

```
Class Student {
    String name;
    int age;
    Student (String n, int a) {
        name = n;
        age = a;
    }
```

Constructors rules

* constructure name must match class name
* no return type
* called automatically during object creation

this keyword

Sometimes parameters names and instance variables names are same

```
class Student {
    String name;
    Student (String name) {
        this.name = name;
    }
}
```

Encapsulation

What is the problem?

Suppose anyone can directly change bank balance

This is dangerous

So OOP says:

Hide data and allow controlled access

This is encapsulation.